

CampusEats — Design: Services, Contracts & Schema

CS543 | Group ID: 02

AMAN KUMAR — 20251651017 | SHRIKANTA NAYAK — 20251651090
ADITYA VERMA — 20251651009 | PRATIK SINHA — 20251651071

1. Capabilities

The capabilities below are derived from the verbs/actions identified in the CampusEats Assignment 1 brief. They represent distinct jobs the system performs.

1. Register
2. Log in
3. Search food
4. Browse menu
5. View food item
6. Place order
7. Cancel order
8. Make payment
9. Track order
10. Manage menu

2. Service Design

CampusEats is divided into four services. Each service owns its own data and exposes operations to other services through a defined contract. The responsible team member is shown for each service.

Service	Responsible Member	Data Owned
Identity Service	Shrikanta Nayak — 20251651090	Users, Roles, Authentication
Order Service	Aditya Verma — 20251651009	Orders, Order Items, Order Status
Catalogue Service	Pratik Sinha — 20251651071	Restaurants, Menus, Food Items
Payment Service	Aman Kumar — 20251651017	Payments, Payment Status

Service design diagram:

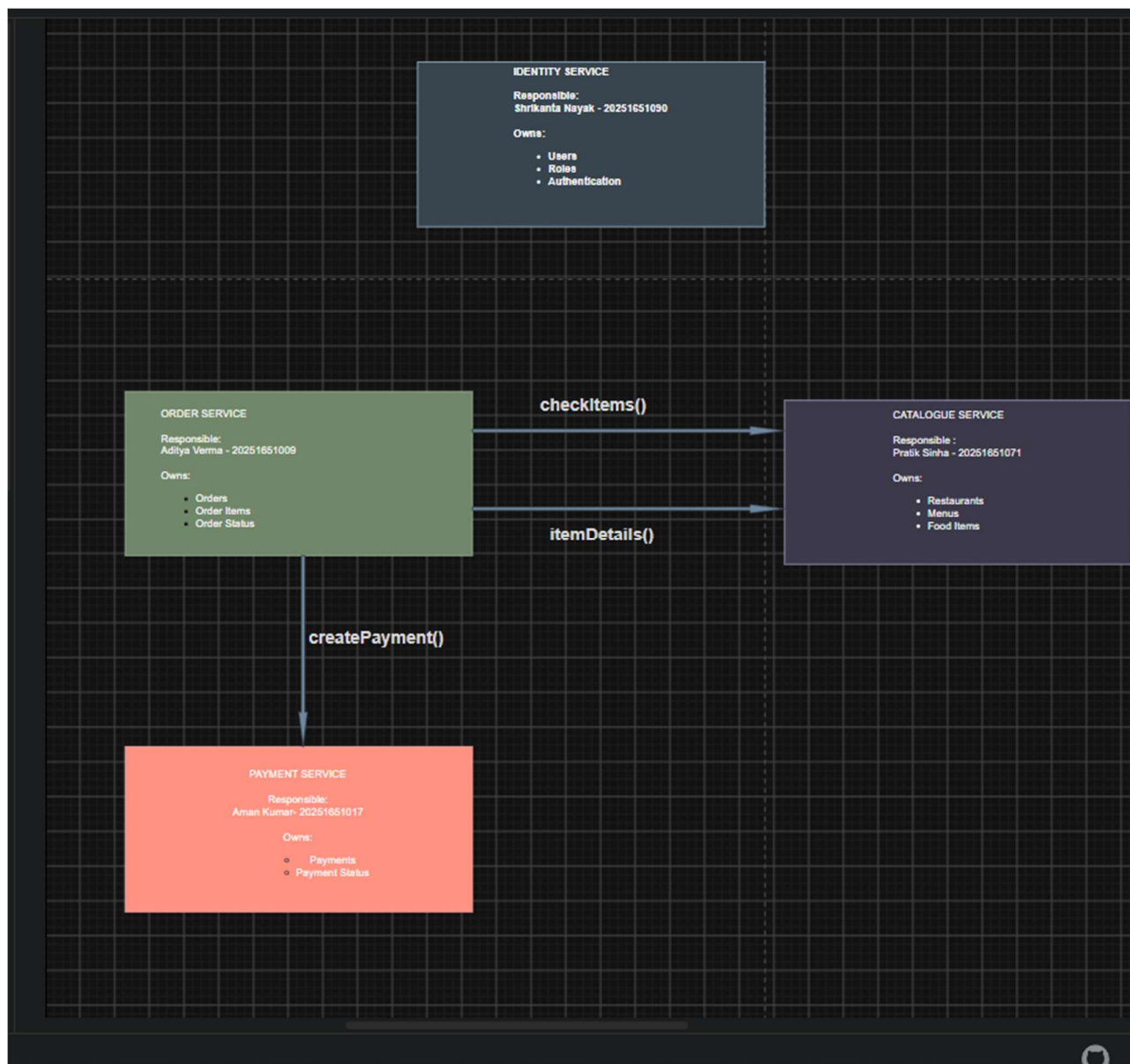


Figure 1. CampusEats service design benchmark.

Cross-service calls shown in the benchmark are labelled with their operations. The Order Service calls the Catalogue Service for item checks/details and calls the Payment Service to create a payment.

3. Service Contracts

A service contract defines the operations that other services may call. It specifies the operation name, input, output, and possible errors without exposing internal database tables, ID formats, SQL, or implementation details.

3.1 Identity Service

Responsible: Shrikanta Nayak — 20251651090

The Identity Service manages user registration, authentication, and user access information.

Operation	Input	Output	Errors
register	Name, email, password, role	Registration result	User already exists; invalid information
login	Email, password	Authentication result	Invalid credentials; account unavailable
getUser	User reference	User information	User not found

3.2 Catalogue Service

Responsible: Pratik Sinha — 20251651071

The Catalogue Service manages restaurants, menus, and food-item information. Other services use it to obtain information about available food.

Operation	Input	Output	Errors
checkItems	List of food-item references	Availability and price information	Item not found; item unavailable
itemDetails	Food-item reference	Food-item details	Item not found
searchFood	Search text	Matching food items	Invalid search
getMenu	Restaurant reference	Restaurant menu	Restaurant not found

3.3 Order Service

Responsible: Aditya Verma — 20251651009

The Order Service manages the food-ordering process, including creating, cancelling, and tracking orders.

Operation	Input	Output	Errors
placeOrder	Customer information, requested items, quantities, payment request	Order confirmation and status	Item unavailable; invalid order; payment failed
cancelOrder	Order reference	Cancellation result	Order not found; order cannot be cancelled
getOrder	Order reference	Order details	Order not found
trackOrder	Order reference	Current order status	Order not found
updateOrderStatus	Order reference, new status	Updated order status	Order not found; invalid status

3.4 Payment Service

Responsible: Aman Kumar — 20251651017

The Payment Service handles payment processing and payment-status information.

Operation	Input	Output	Errors
createPayment	Order information, amount, payment details	Payment result and status	Payment declined; invalid payment information
getPaymentStatus	Payment reference	Current payment status	Payment not found

Contract Boundary

The contracts expose only service operations and their inputs, outputs, and errors. Callers do not need to know which database is used, which tables exist, how data is stored, the SQL queries used internally, primary-key formats, or internal transaction handling.

4. placeOrder — Full Specification

4.1 Purpose

placeOrder creates a new food order for a customer using the requested food items and quantities. It coordinates with the Catalogue Service to verify the requested items and with the Payment Service to process the payment.

4.2 Input

placeOrder(customer, items, paymentDetails)

The input contains:

- Customer — information identifying the customer placing the order.
- Items — the food items requested and their quantities.
- Payment Details — information required to process the payment.

4.3 Output

On successful completion, the operation returns an Order Result containing:

- Order confirmation
- Order status
- Total amount
- Payment status

Example:

```
Order Result
Order: Confirmed
Status: Placed
Total Amount: <calculated amount>
Payment Status: Successful
```

4.4 Error Cases

Error	Meaning
CustomerNotFound	The customer cannot be identified.
ItemNotFound	One or more requested food items cannot be found.
ItemUnavailable	One or more requested items are currently unavailable.
InvalidQuantity	A requested quantity is invalid.
InvalidOrder	The order information is incomplete or invalid.
PaymentFailed	Payment processing was unsuccessful.
OrderCreationFailed	The order could not be completed.

4.5 Processing

11. Validate the customer and order request.
12. Check the requested food items with the Catalogue Service.
13. Verify item availability and required information.
14. Calculate the order total.
15. Request payment through the Payment Service.
16. If payment succeeds, create the order.
17. Return the order confirmation and status.

4.6 Details Hidden from Callers

The caller of placeOrder does not need to know how the operation is implemented internally. The following details remain hidden:

- Database tables
- SQL queries
- Primary-key formats
- How orders are stored
- How order items are stored
- How item information is stored
- Internal payment implementation
- Internal database transactions
- Internal communication mechanisms
- Internal error-handling logic

The caller interacts only with the placeOrder contract: Input → placeOrder → Output / Error. This keeps the Order Service independently implementable while exposing only the behavior required by other services.

5. Database Schema

The database schema extends the same service boundaries used in the service design. Each table has a single owning service. Cross-service references are represented as references between services and do not transfer ownership of the referenced data.

5.1 Data Ownership

Service	Tables Owned
Identity Service — Shrikanta Nayak	roles, users
Catalogue Service — Pratik Sinha	restaurants, menus, food_items
Order Service — Aditya Verma	orders, order_items
Payment Service — Aman Kumar	payments

5.2 Identity Service — roles

Column	Type	Key / Reference
role_id	INTEGER	PK
role_name	VARCHAR(50)	UNIQUE

5.3 Identity Service — users

Column	Type	Key / Reference
user_id	INTEGER	PK
name	VARCHAR(100)	
email	VARCHAR(150)	UNIQUE
password	VARCHAR(255)	
role_id	INTEGER	FK → roles.role_id

5.4 Catalogue Service — restaurants

Column	Type	Key / Reference
restaurant_id	INTEGER	PK
name	VARCHAR(150)	
location	VARCHAR(200)	

5.5 Catalogue Service — menus

Column	Type	Key / Reference
menu_id	INTEGER	PK
restaurant_id	INTEGER	FK → restaurants.restaurant_id
name	VARCHAR(150)	

5.6 Catalogue Service — food_items

Column	Type	Key / Reference
item_id	INTEGER	PK
menu_id	INTEGER	FK → menus.menu_id
name	VARCHAR(150)	
description	TEXT	
price	DECIMAL(10,2)	
available	BOOLEAN	

5.7 Order Service — orders

Column	Type	Key / Reference
order_id	INTEGER	PK
customer_id	INTEGER	Reference → Identity.users
status	VARCHAR(30)	
total_amount	DECIMAL(10,2)	
created_at	TIMESTAMP	

5.8 Order Service — order_items

Column	Type	Key / Reference
order_item_id	INTEGER	PK
order_id	INTEGER	FK → orders.order_id
item_id	INTEGER	Reference → Catalogue.food_items
quantity	INTEGER	
price	DECIMAL(10,2)	

5.9 Payment Service — payments

Column	Type	Key / Reference
payment_id	INTEGER	PK
order_id	INTEGER	Reference → Order.orders
amount	DECIMAL(10,2)	
status	VARCHAR(30)	
payment_method	VARCHAR(30)	
created_at	TIMESTAMP	

5.10 ER Relationships

- roles 1:N users
- restaurants 1:N menus
- menus 1:N food_items
- orders 1:N order_items
- orders.customer_id → Identity Service users (cross-service reference)
- order_items.item_id → Catalogue Service food_items (cross-service reference)
- payments.order_id → Order Service orders (cross-service reference)

5.11 Boundary Rule

No table belongs to two services. Identity owns user data, Catalogue owns food and menu data, Order owns order data, and Payment owns payment data. Cross-service references identify related data owned by another service but do not create shared table ownership.

5.12 Equivalent CREATE TABLE Sketch

```
-- Identity Service
CREATE TABLE roles (
    role_id INTEGER PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL UNIQUE
);
```

```

CREATE TABLE users (
    user_id INTEGER PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role_id INTEGER,
    FOREIGN KEY (role_id) REFERENCES roles(role_id)
);

-- Catalogue Service
CREATE TABLE restaurants (
    restaurant_id INTEGER PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    location VARCHAR(200)
);

CREATE TABLE menus (
    menu_id INTEGER PRIMARY KEY,
    restaurant_id INTEGER NOT NULL,
    name VARCHAR(150) NOT NULL,
    FOREIGN KEY (restaurant_id) REFERENCES restaurants(restaurant_id)
);

CREATE TABLE food_items (
    item_id INTEGER PRIMARY KEY,
    menu_id INTEGER NOT NULL,
    name VARCHAR(150) NOT NULL,
    description TEXT,
    price DECIMAL(10,2) NOT NULL,
    available BOOLEAN NOT NULL DEFAULT TRUE,
    FOREIGN KEY (menu_id) REFERENCES menus(menu_id)
);

-- Order Service
CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    status VARCHAR(30) NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    created_at TIMESTAMP NOT NULL
);

CREATE TABLE order_items (
    order_item_id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL,
    item_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

-- Payment Service
CREATE TABLE payments (
    payment_id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(30) NOT NULL,
    payment_method VARCHAR(30) NOT NULL,
    created_at TIMESTAMP NOT NULL
);

```

6. Service Validation

Each CampusEats service is checked against the five properties of a service: reachable, self-contained, has a contract, independent, and loosely coupled. Where a property is not fully satisfied, the issue and a possible fix are identified.

Service	Reachable	Self-contained	Has Contract	Independent	Loosely Coupled	Issue / Fix
Identity Service	Yes	Yes	Yes	Yes	Yes	No major issue. Keep authentication and user data behind its contract.
Catalogue Service	Yes	Yes	Yes	Yes	Yes	No major issue. Expose only catalogue operations to other services.
Order Service	Yes	Yes	Yes	Yes	Partly	It depends on Catalogue and Payment for order completion. Keep these dependencies limited to checkItems/itemDetails and createPayment contracts.
Payment Service	Yes	Yes	Yes	Yes	Partly	It is called by Order during placeOrder. Keep the dependency limited to createPayment and getPaymentStatus; hide payment implementation details.

6.1 Validation Notes

Reachable: Each service is intended to be accessible through its exposed operations.

Self-contained: Each service owns its own data and implementation boundary.

Has a Contract: Each service exposes defined operations with inputs, outputs, and errors.

Independent: No service owns another service's data; each can maintain its internal implementation separately.

Loosely Coupled: Cross-service dependencies are limited to explicit operations. Order has dependencies on Catalogue and Payment, so those calls should remain narrow and contract-based.

6.2 Overall Assessment

The four services satisfy the core service properties. The main coupling occurs in the Order Service because placing an order requires information from the Catalogue Service and payment processing from the Payment Service. This is acceptable when the dependencies remain limited to the published contracts and no internal data or implementation details are shared.

Conclusion

The CampusEats design defines capabilities, service boundaries, contracts, the central placeOrder operation, and a database schema that follows the ownership boundary. The validation confirms that the services are reachable,

self-contained, contract-based, and independently owned, while cross-service dependencies are kept limited to explicit operations.